

Q1

1a

I choose a **differential-drive system**. It is **simple and stable**, and allows smooth curved turns so the vacuum at the back can pass over the brushed area without spinning in place. The robot is **non-holonomic**, since it cannot move sideways without changing heading. And its configuration space is $SE(2) = \{(x, y, \theta)\}$.

1b:

If the via-point order is fixed, plan **separate shortest collision-free paths** (e.g., A* or Dijkstra) from Start to the first via point, between successive via points, and finally to Finish, then join them.

If the order is free, **solve a TSP on pairwise path costs to find the best order before planning**.

1c:

A **quintic-polynomial time-scaling function** is chosen because it ensures zero velocity and acceleration at the start and end, giving smooth c^2 -continuous motion. For multiple via points, piecewise quintic segments are used so the robot passes smoothly through each point without stopping, avoiding jerky motion that could affect cleaning.

1d:

Because the vacuum is at the back, the path should avoid **in-place rotations**. Instead, use **smooth curves with limited turning radius**, and add **short straight compensation segments** after turns so the vacuum path completely overlaps the brushed area, preventing uncollected water.

1e:

Boustrophedon decomposition coverage method can be used. The free space can be divided into obstacle-free cells, and every cell is covered using a back-and-forth sweeping pattern. Collision-free transition paths are then planned between cells to avoid obstacles and restricted zones while ensuring full-area coverage.

Q2

2a

In `load_targets()`, I first open the ROS2 bag file using `sqlite3`, and read all messages published on the `sensor_msgs/JointState` topic. From each message, I extract the first five joint angles, corresponding to the five actuated joints of the Youbot arm. These joint configurations are stored in a $5 \times N$ matrix.

For each joint configuration, I then use the provided **KDL forward-kinematics function** to compute the end-effector pose. The result is a sequence of 4×4 homogeneous transformation matrices, each representing a Cartesian checkpoint. These checkpoints are later used for shortest-path planning.

2b

In `get_shortest_path()`, the Cartesian coordinates of all checkpoints are obtained by extracting the position elements from their 4×4 transformation matrices. The end-effector pose corresponding to the zero joint configuration is treated as the starting point.

All permutations of the checkpoint indices are generated, and for each permutation the total Euclidean path length is computed as the sum of the distances between successive checkpoints, including the distance from the initial pose to the first checkpoint. The permutation that yields the minimum total distance is selected as the shortest Cartesian path, and the associated index sequence is returned.

2c

The `decoupled_rot_and_trans()` function interpolates between two checkpoint transforms by separating rotation and translation. The translation vectors are linearly interpolated. The rotation matrices are converted to quaternions, interpolated using SLERP, and converted back to rotation matrices. Each interpolated rotation and translation is then combined into a 4×4 homogeneous transform to generate `num_points` intermediate poses.

The `intermediate_tfs()` function applies this interpolation to each consecutive pair of checkpoints in the shortest-path order. For the first segment all poses are kept, while for later segments the first pose is removed to avoid duplication at the boundaries. The resulting poses are concatenated into a single continuous sequence of 4×4 transforms along the full path.

2d:

The `ik_position_only()` function implements an iterative position-only inverse kinematics solver. The target position is taken from the translation part of the 4×4 pose. For each iteration, the current end-effector position is computed from forward kinematics and subtracted from the target to obtain a 3D position error. The position part of the Jacobian (first three rows) is obtained, and its pseudoinverse is used to compute a joint update scaled by a constant step size. Iteration terminates when the error falls below a tolerance or the maximum number of iterations is reached. Orientation is not constrained.

The `full_checkpoints_to_joints()` function converts the full set of checkpoint transforms, including intermediate points, into a corresponding joint trajectory. Each pose is processed sequentially using `ik_position_only()`. The solution from the previous pose is used as the initial guess for the next pose so that the resulting joint trajectory remains continuous.

Q3

3a

The manipulator only needs 3 translational DOF (X, Y, Z): X follows the conveyor motion, Y reaches any position across the 20 cm belt, and Z performs pick-and-place. No rotational DOF are required because the candies are quasi-spherical and orientation-independent, enabling a simple and fast design.

3b:

A parallel Delta robot is selected because it is widely used for high-speed pick-and-place on conveyors. Its low moving mass enables high acceleration and fast response, while 3 translational DOF are sufficient for the 20 cm workspace without redundancy. A vacuum suction cup is used as the end-effector, since the candies are light and quasi-spherical, and orientation control is unnecessary.

3c:

Brushless DC (or AC) servo motors are selected for the **Delta robot**, as they provide **high power density, fast dynamic response, and accurate position control**, which are essential for high-speed pick-and-place. The motors are base-mounted and drive the links through lightweight belt or gear transmission to minimise moving inertia and improve efficiency.

3d:

Rotary encoders are installed at the **joints** to provide position and velocity feedback for **closed-loop control**. A **fixed RGB industrial camera** above the conveyor identifies the target candy by colour and determines its planar position. As the conveyor height is known, no 3D vision is required, keeping the system minimal.

3e:

During long-term high-speed repetitive operation, the bearings and transmission components (belts or gears) are the most wear-prone parts due to frequent acceleration and cyclic loading. Wear can be reduced by using durable materials, regular lubrication, smoother motion profiles, and base-mounted actuators to minimise moving inertia.

3f

Without modifying the conveyor, colour detection and localisation are still performed by the vision system. A **small load-cell** is integrated at the end-effector to weigh the candy after pickup. If the measured weight exceeds the set threshold, it is placed in the basket; otherwise it is discarded. This adds only one essential sensor while keeping the system minimal.

Q4:

4a:

In this task, we need to compute the Jacobian at the Centre of Mass (CoM) of a given link of the KUKA iiwa14 manipulator. To achieve this, I first construct the forward kinematic transformations from the base to each joint. At every joint, I extract the joint axis direction z_i and the joint position p_i expressed in the base frame, since these describe how each joint influences the system motion. By further propagating the forward kinematics to the target link and applying the known offset from the link frame to its CoM, the CoM position in the base frame, p_{com} , can be obtained.

Because all joints of the iiwa14 are revolute, the linear velocity contribution of each joint to the CoM can be expressed using a cross product between the joint angular velocity axis and the position vector from the joint to the CoM. Therefore, the linear part of the CoM Jacobian is

$$J_{v,i} = z_i \times (p_{\text{com}} - p_i) \quad (1)$$

and the angular part is simply

$$J_{\omega,i} = z_i \quad (2)$$

By stacking the linear and angular velocity components of all joints, a full 6×7 CoM Jacobian matrix is obtained. The first three rows correspond to linear velocity, and the last three rows correspond to angular velocity. This Jacobian forms the basis for subsequent dynamic modelling, including the computation of the inertia matrix $B(q)$, the Coriolis/centrifugal term $C(q, \dot{q})\dot{q}$, and the gravity term $g(q)$, which together fully

describe the dynamic behaviour of the manipulator.

4b

In this task, I implemented the inertia matrix $B(q)$ of the KUKA iiwa14 manipulator based on rigid-body dynamics. The total kinetic energy of the robot can be written as the sum of the translational and rotational kinetic energies of all links. By expressing the kinetic energy in quadratic form and rearranging the expression, the inertia matrix can be written as

$$B(q) = \sum_i \left[m_i J_{v,i}^T J_{v,i} + J_{\omega,i}^T R_i I_i R_i^T J_{\omega,i} \right] \quad (3)$$

where

- m_i is the mass of link i
- $J_{v,i}$ is the linear velocity Jacobian of the link CoM
- $J_{\omega,i}$ is the angular velocity Jacobian of the link
- I_i is the inertia tensor of the link expressed in its own frame
- R_i is the rotation matrix from the link frame to the base frame

The implementation follows these steps:

1. Use the CoM Jacobian computed in 4(a) to obtain both the linear and angular velocity mappings of each link CoM with respect to the joint velocities.
2. Compute the forward kinematics of each link to obtain its orientation and the corresponding rotation matrix R_i .
3. Transform the inertia tensor I_i from the link frame into the base frame.
4. Accumulate the translational inertia term $m_i J_{v,i}^T J_{v,i}$ and the rotational inertia term $J_{\omega,i}^T R_i I_i R_i^T J_{\omega,i}$ for all links to form the final 7×7 inertia matrix $B(q)$.

The structure of the implementation follows the standard rigid-body dynamics formulation and matches the KDL implementation conceptually.

4c

In this task, the goal is to compute the Coriolis and centrifugal vector

$$C(q, \dot{q})\dot{q} \quad (4)$$

where $B(q)$ is the joint-space inertia matrix with elements B_{ij} .

According to Lagrangian dynamics, the Christoffel symbols are defined as

$$c_{ijk} = \frac{1}{2} \left(\frac{\partial B_{ij}}{\partial q_k} + \frac{\partial B_{ik}}{\partial q_j} - \frac{\partial B_{jk}}{\partial q_i} \right) \quad (5)$$

where $\frac{\partial B_{ij}}{\partial q_k}$ denotes the partial derivative of B_{ij} with respect to the k -th joint variable.

The i -th component of the Coriolis/centrifugal vector is then given by

$$C_i = \sum_j \sum_k c_{ijk} \dot{q}_j \dot{q}_k \quad (6)$$

In the implementation, I first compute the inertia matrix $B(q)$ using the function developed in 4(b). The required partial derivatives are then approximated numerically:

$$\frac{\partial B_{ij}}{\partial q_k} \approx \frac{B_{ij}^+ - B_{ij}}{\varepsilon} \quad (7)$$

where B^+ is the inertia matrix evaluated at

$$q^+ = q, \quad q_k^+ = q_k + \varepsilon \quad (8)$$

The resulting derivatives are substituted into the Christoffel symbol definition, and the double summation is used to obtain each element of the vector $C(q, \dot{q})\dot{q}$.

This numerical differentiation approach avoids symbolic computation while remaining fully consistent with the theoretical definition. The computed result was compared against the KDL `JntToCoriolis()` output, and the two matched numerically, confirming the correctness of the implementation.

4d

The total gravitational potential energy of the manipulator can be written as

$$U(q) = \sum_i m_i g^T p_{\text{com},i}(q) \quad (9)$$

where

- m_i is the mass of link i
- g is the gravitational acceleration vector
- $p_{\text{com},i}(q)$ is the position of the CoM of link i expressed in the base frame

From Lagrangian dynamics, the corresponding generalised gravitational force is obtained as

$$g(q) = \frac{\partial U(q)}{\partial q} \quad (10)$$

Substituting the potential energy expression yields

$$g(q) = \sum_i m_i J_{v,i}^T g \quad (11)$$

where $J_{v,i}$ is the linear velocity Jacobian of the CoM of link i , i.e. the upper three rows of the CoM Jacobian returned by `get_jacobian_centre_of_mass()`.

The implementation follows the structure:

$$G = 0 \quad (12)$$

for each link i :

$$G += m_i (J_{v,i}^T g) \quad (13)$$

resulting in a 7×1 joint gravity compensation vector $g(q)$.

The computed gravity term was numerically compared against the KDL `JntToGravity()` output, and the two results matched, confirming the correctness of the implementation.

Q5:

The **Huygens–Steiner theorem**, also known as the parallel axis theorem, describes the relationship between the moment of inertia of a rigid body about any given axis and its moment of inertia about a parallel axis passing through its centre of mass. It can be written as

$$I = I_c + md^2 \quad (14)$$

where

- I is the moment of inertia about the chosen axis,
- I_c is the moment of inertia about a parallel axis through the centre of mass,
- m is the mass of the body, and
- d is the distance between the two axes.

The theorem assumes that the body is rigid, the mass distribution is fixed, and the two axes are parallel with a known centre-of-mass location.

Derivation:

The derivation is obtained by integrating the contribution of all mass elements. The position of each mass element is decomposed into its coordinate relative to the centre of mass and the offset distance between the two axes. When the expression is expanded, the cross-term involving the centre-of-mass position vanishes because the vector sum of all mass elements about the centre of mass is zero. The remaining terms lead directly to

$$I = I_c + md^2. \quad (15)$$

Importance in robotics:

This theorem is fundamental in robot dynamics. Link inertial parameters are typically defined about the link centre of mass, whereas dynamic formulations (e.g., Newton–Euler or Lagrangian methods) require moments of inertia about the joint axes. The Huygens–Steiner theorem allows inertia tensors to be transformed between different

Q6

Forward dynamics refers to the problem of determining the motion of a robot given the applied joint torques and the dynamic model. The robot dynamics are described by

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) \quad (16)$$

Given the joint torques τ , the joint accelerations $\ddot{q}$ are obtained by inverting the equation, and joint velocities and positions are then found through numerical integration. This process therefore maps from **torque** to **motion**. Forward dynamics is mainly used in simulation, motion prediction, physical modelling, and validation of control algorithms.

The main difficulties arise from the strong nonlinearity and coupling in the dynamics, the need for numerical integration, and the sensitivity to modelling inaccuracies.

Inverse dynamics

Inverse dynamics is the opposite problem: given a desired motion trajectory $q(t)$, $\dot{q}(t)$, $\ddot{q}(t)$, the goal is to compute the joint torques required to realise this motion. This represents the mapping from **motion** to **torque**. It is widely used in torque-based control, trajectory tracking, load estimation, and energy analysis.

The main challenges are its high sensitivity to model parameters such as masses, inertias, and friction. Any modelling error directly affects the computed torques. In addition, for redundant manipulators, the torque solution may not be unique.

Summary

Forward dynamics is mainly associated with **simulation and prediction**, whereas inverse dynamics is mainly used for **control and execution**. Together, they form the foundation of robot dynamic modelling and control design.

Q7:

7a:

The bag file `cw2q7/bag/data_ros2/data_ros2.db3` was successfully loaded and parsed. It contains **a single topic**, `/iiwa/EffortJointInterface_trajectory_controller/command`, which keeps publishes messages of type `trajectory_msgs/JointTrajectory`.

Only one `JointTrajectory` message is recorded in the bag file. This message contains **three JointTrajectoryPoints corresponding** to desired joint positions at **10 s, 20 s, and 30 s** from the start of the trajectory. The velocity and acceleration fields are set to zero, and the effort field is empty.

Therefore, the bag file provides only joint position trajectory information, while dynamic quantities such as joint accelerations must be computed separately.

7b

This problem is an **inverse dynamics problem**.

Given the joint position trajectory, the corresponding joint velocities and accelerations can be derived. By combining this information with the robot's dynamic model, the required joint torques or dynamic terms needed to produce the specified motion can be computed. This direction of computation is opposite to that of forward dynamics, which determines the motion of the robot given known joint torques. Therefore, this problem should be classified as an inverse dynamics problem.

7c

Using the file-reading method implemented in Q7(a), the joint position data of each `JointTrajectoryPoint` can be extracted. The trajectory consists of multiple joint configurations, each corresponding to the robot manipulator's joint posture at different time instances.

A `JointTrajectory` publisher was then created to publish this information to the `/iiwa_controller/joint_trajectory` topic. The trajectory points are published sequentially through a timer callback, with one `JointTrajectoryPoint` being sent at each iteration to command the joint motion. Once all trajectory points have been published, the publishing process terminates.

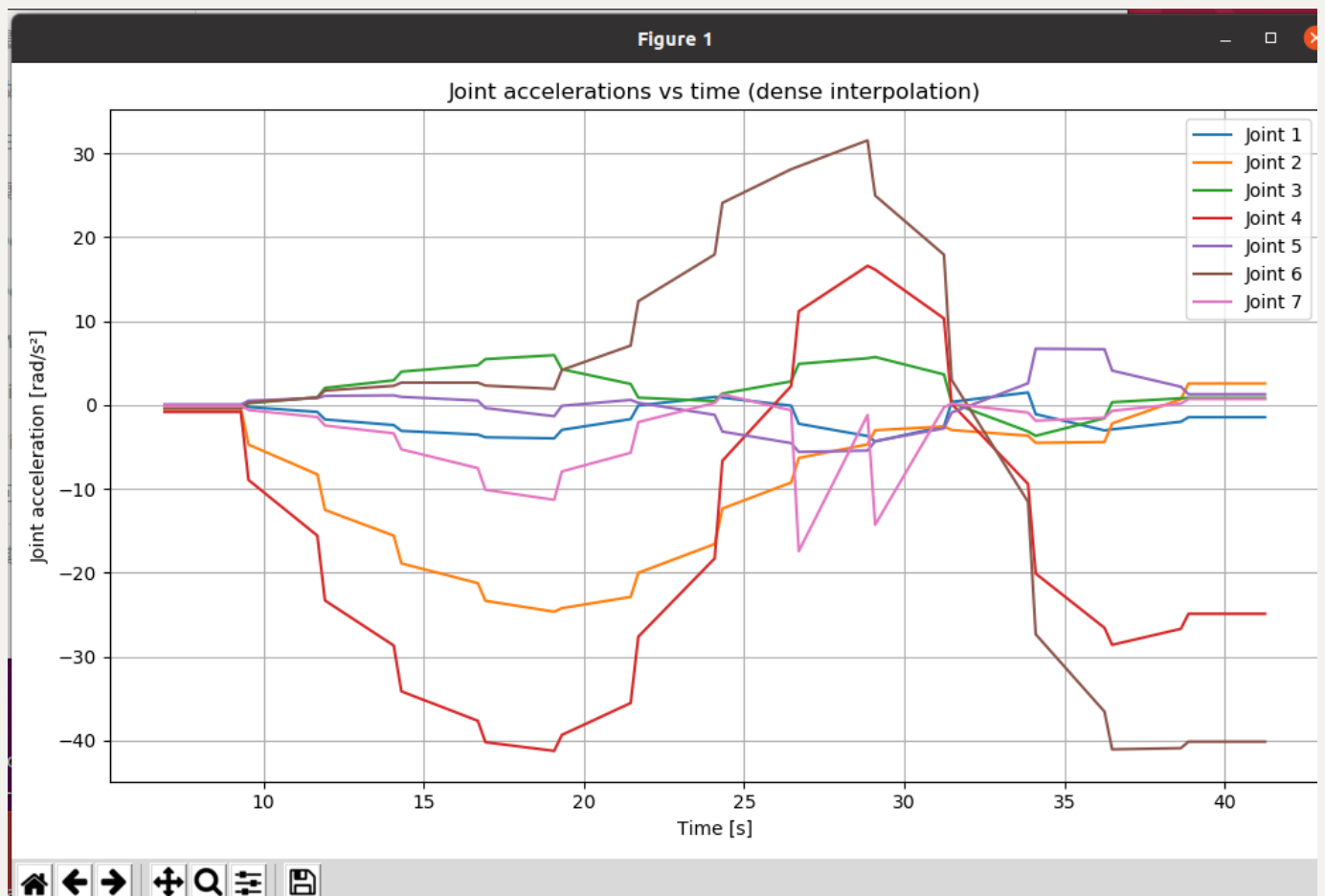
7d

This question subscribes to the `/joint_states` topic to obtain joint information in real time during trajectory execution. When a `JointState` message is received, the **joint velocity vector** and **joint velocity vector** are extracted, and the methods `get_B()`, `get_C_times_qdot()`, and `get_G()` implemented in Question 4 are used to compute the inertia matrix $B(q)$, the Coriolis and centrifugal term $C(q, \dot{q})\dot{q}$, and the gravity term $G(q)$, respectively. Based on the spatial dynamics equation, the joint accelerations can be obtained as:

$$\ddot{q} = B(q)^{-1}(-C(q, \dot{q})\dot{q} - G(q)) \quad (17)$$

According to this equation, the joint accelerations $\ddot{q}$ can be computed.

7e



Based on the formulation in Question 7d, the joint accelerations $\ddot{q}$ can be obtained. By plotting the joint accelerations as a function of time in Python, Figure 1 is produced. As shown in the figure, the horizontal axis represents time (s) and the vertical axis represents joint acceleration (rad/s²), with different curves corresponding to the seven joints of the robotic manipulator.

From the figure, it can be observed that the acceleration curves of some joints are not completely smooth. This is mainly due to the limited number of trajectory points and the relatively sparse temporal interpolation, which introduces discretization effects in the numerical computation of joint velocities and accelerations. Nevertheless, the overall trend of the joint accelerations is clearly visible and is sufficient to reflect the dynamic characteristics of each joint during trajectory execution; therefore, this does not affect the analysis or the conclusions.

In addition, due to the continuous effect of the gravity term $G(q)$, the joint accelerations do not necessarily converge to zero even when the joints are in a stationary state. Despite this, the overall acceleration trends remain clear and are able to capture the characteristics of the manipulator during the execution of the trajectory.